Multi-Agent Debate for Quantitative Trading Signal Discovery

CS153 Final Project · Stanford · June 2026

Patrick Flanagan

Contents

Multi-Agent Debate for Quantitative Trading Signal Discovery

Patrick Flanagan · CS153 Final Project · Stanford · June 2026

Project Goal

Build an LLM agent system that helps my systematic quantitative-equity strategy find new candidate factor ideas.

What kind of trading is this

I run a **systematic quantitative-equity strategy**. Every trading day, a computer system I built ranks the universe of ~3,000 US large-cap stocks, buys the top of the list (long), and simultaneously sells short the bottom (short). It holds for a few days, re-ranks the universe, and repeats. Because the long and short sides are equal-sized, the

strategy is roughly **market-neutral** — my portfolio's beta to the S&P 500 is **-0.058**, effectively zero. The bet is that the spread between top-ranked and bottom-ranked stocks stays positive after costs, in any kind of market.

What factors are

The daily ranking is built from **factors** — formulas that produce a number for each stock each day. Classic example: 12-month price momentum. The hypothesis is that ranking stocks by their return over the previous 12 months will predict which stocks outperform over the next window. My strategy uses **several thousand factors**. A daily-updating machine learning model takes all of their values for every stock and combines them into one final composite score that drives the long/short ranking.

The strategy's quality is bottlenecked on the quality of the factor library. Better factors → better ranking → better returns.

Why this is hard, and where the LLM fits

The metric I care about is **Sharpe ratio** — annualised return divided by annualised volatility. A Sharpe of 1 is fine, 2 is rare, and 3 is the kind of thing institutional investors pay attention to. My deployed configuration runs at **Sharpe 3.33** over 4.8 years out of sample. Getting there is hard because *most apparent factors are noise* — they look great on historical data they were fit on and fail on data they weren't. To keep finding edges before existing ones decay, the strategy needs a constant stream of new factor ideas to test.

Generating those is the creative bottleneck. This project is an LLM agent system that generates them, with the failure modes of single-LLM ideation explicitly designed out.

What I built and why

So the obvious first move is to just ask an LLM to propose new factor ideas. When I tried that with a single LLM, two things went wrong every time:

- **Every proposal sounded like momentum.** Even with high temperature and "be creative" prompting, a single LLM collapses onto the same handful of textbook ideas.
- **The LLM cheerfully approved its own work.** When the same agent proposed a factor and then reviewed it, the approval rate was close to 100%. That meant my "filter" wasn't filtering anything — proposals with subtle lookahead bias, ignored transaction costs, or in-sample overfitting were sailing through.

So I built a four-stage debate system to test the claim that **mandate-diversified Proposers plus a structurally separated Critic produce better candidate factors than a single LLM asking itself.**

The system, plain English

Four LLM agents, each with their own job:

1. **Proposer.** Five copies of it run in parallel. Each copy gets a different *mandate*: textbook, cross-domain, contrarian, interaction, orthogonal. The system prompt tells each one *"you are one of 5 — do not hedge, commit to your mandate."* That single instruction is what stops everyone collapsing onto momentum.
2. **Critic.** Reviews the proposals in batches. Crucially, the Critic only sees the proposal's name, rationale, and computation description — *never* the Proposer's chain of thought or which slot it came from. That structural separation is what stops the anchoring problem that makes single-agent self-review useless.
3. **Revisor.** Every "revise" verdict from the Critic gets fixed in a single batched LLM call. One call, ~20% of the slate recovered. Cheap.
4. **Ranker.** Only fires when the Critic approves more than the slate cap. Picks for *diversity first*, *relevance second*, *simplicity as tie-breaker*.

The full flow:

```
research challenge
       │
       ▼
Proposer × 5 parallel slots ——> ~25 raw candidates
(textbook / cross_domain / contrarian / interaction / orthogonal)
       │
       ▼
Critic in batches → {approve, revise, reject}
       │
       ▼
Revisor (one batched call across every "revise")
       │
       ▼
Backfill if approved < target
       │
       ▼
Ranker picks top-k if approved > max
       │
       ▼
final slate of candidate factors → fed into the live strategy's
factor library and downstream production pipeline
```

Each of the four agents has its own system prompt; they're all in `prompts/` as standalone text files. Edit any of them without touching the Python.

A real debate, walked through

The easiest way to see what the system actually does is to look at one debate end-to-end. The repo has three real (redacted) production transcripts in `transcripts/`; this is the most representative one.

The challenge given to the agents: a research scenario where the live ranker had over-rotated into a crowded earnings-growth × momentum trade right before a volatility-regime shift. Propose new candidate factors that would have given the ranker more diverse information on a day like that.

What happened:

- The five Proposer slots produced 29 raw candidates between them.
- The Critic approved 10, sent 10 back for revision, rejected 0.
- The Revisor fixed 4 of the 10 revise verdicts. Final slate: 14.
- 13 LLM calls total. Wall clock: 8 minutes.

The interesting part — what the Critic actually caught. One of the textbook-slot proposals was a 60-day rolling-alpha factor. The Critic flagged it:

> *"The proposed derivation uses a 60-day rolling regression on the full window including the target date. This embeds lookahead — the regression coefficient for day t would be fitted on data including day t's return."*

And gave specific revision guidance:

> *"Shift the rolling window to end at t-1, not t. Use the coefficient from the regression on days [t-60, t-1] to compute the alpha for day t."*

That's the kind of catch a careful human reviewer would make. A single LLM proposing and reviewing its own work would not. The Revisor took the guidance, fixed the factor, and the revised version was approved.

You can replay this exact debate locally with no API key: `python run_demo.py --dry-run`.

Did the system work?

Three numbers tell the story. **1,214** raw LLM proposals went in. **448** of them (about **37%**) survived the Critic + Revisor + a validation gate and were added to the live factor library. Those 448 then seeded **800** promoted factors that the trading strategy actually uses.

The first transition shrinks. The second grows. The shrink is the architecture doing real work — a single LLM agent that reviews its own work approves at near 100%, but this system rejects roughly two thirds of raw proposals. That rejection is the adversarial Critic catching the lookahead bugs, the duplicates of things already in the library, and the proposals that double down on whatever direction just failed.

Per-debate runtime is steady:

Metric	Min	Median	Max
Raw proposals per debate	16	27	35
Approved per debate	11	14	15
LLM calls per debate	8	13	16
Wall clock (seconds)	270	535	1,191

Notice LLM call count stays roughly constant as raw-proposal count grows. That's the

batched Critic and single-call Revisor working as designed — they don't scale linearly with the number of proposals.

The strategy's factor library now consists of **783 LLM-proposed factors and 89 hand-curated ones** — about 90% authored by this system.

Where the LLM system sits in the trading strategy

The debate system sits at the top of the strategy's research pipeline. It produces candidate factor ideas. Those ideas then run through several downstream stages of the production trading pipeline before any of it touches the live portfolio. The LLM is not solely responsible for the portfolio numbers. The performance table below reflects the integrated behaviour of the full pipeline.

The strategy's actual performance

Out-of-sample evaluation window: **2021-07-16 → 2026-05-05** (1,206 trading days, **4.8 years**). The deployed configuration is **K = 25 at 1.5× gross leverage**.

K	Unlevered CAGR	Unlevered Sharpe	Unlevered Max DD	1.5× CAGR	1.5× Sharpe	1.5× Max DD	1.5× total return
5	84.1%	2.77	-15.1%	142.7%	2.74	-25.2%	69.6×
10	87.6%	3.16	-12.5%	141.1%	3.02	-20.8%	67.5×
25	**81.1%**	**3.43**	**-10.0%**	**134.0%**	**3.33**	**-15.7%**	**58.4×**
50	70.9%	3.45	-8.6%	119.9%	3.44	-12.9%	43.4×
100	58.8%	3.44	-7.3%	98.6%	3.44	-10.8%	26.6×
250	42.2%	3.32	-6.3%	70.4%	3.40	-9.3%	12.8×

The deployed row — K=25 at 1.5× leverage — is the headline. On that configuration: Newey-West t-statistic **8.17**, SPY beta **-0.058** (effectively market-neutral), 5-day non-overlapping hit rate **71.8%**, **48 of 59 OOS months positive**, **$1 → $58.40** over the 4.8 years.

Sharpe is essentially flat across K = 10 to 250 at ~3.3 to 3.45, which means the signal isn't living in one weird corner of the rank distribution. CAGR scales sub-linearly with leverage (134% at 1.5× versus 81% unlevered), which is what you'd expect once larger drawdowns start hurting compounding.

The cumulative equity curve, drawdown trajectory, and a monthly-returns breakdown are in **`demo.ipynb`** at the bottom of the notebook — they're worth a look if you haven't seen them.

What I learned about LLM agents

1. **Mandate diversification matters more than temperature.** I tried high-temperature single-Proposer setups first. Everything still collapsed to momentum. Five parallel Proposers, each told *"you are one of five — do not hedge"*, produce candidate sets that span the signal space. It's not a sampling-temperature problem; it's a prompt-structure problem.
2. **Structurally separating the Critic from the Proposer is the load-bearing decision.** Giving the Critic only `(name, rationale, derivation)` — never the Proposer's chain of thought, never which slot the proposal came from — is what changes the approval rate from ~100% to 37%. Without that separation, the Critic anchors on the Proposer's framing.
3. **A batched Revisor is the cheapest filter you can add.** One LLM call, ~20% of the slate recovered. Re-proposing rejected candidates from scratch would cost five times as much.
4. **The Ranker fires occasionally but is load-bearing when it does.** On clean-pass days where the Critic approves more than the slate cap, the Ranker is the only thing preventing the slate from collapsing onto whichever Proposer slot got lucky that round.
5. **The reliability engineering around the agent logic is harder than the agent logic itself.** The continuous-debate cron has logged 9,592 round-starts; only 5 completed cleanly. The agent code works — the wrapper around it (rate-limit handling, dependency pinning, paid-tier fallback, alerting when the cron drifts) is where time disappears in production. The 1,214 proposals above were produced in a ~3-day window in early April before the cron drifted; reliability is what's keeping the system from producing another batch right now.

What I'd do next

- **I don't have a formal single-agent ablation yet.** The 37% materialisation rate strongly suggests the architecture is earning its complexity, but a clean three-way head-to-head — single-agent vs. no-Critic vs. full debate on identical challenges — would actually pin down how much each piece is contributing.

How to run this

All commands verified end-to-end in a fresh Python 3.13 venv.

```
# Install
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# A — replay a real recorded production debate (no API key needed)
python run_demo.py --dry-run

# B — open the walkthrough notebook (charts + tables pre-computed)
```

```
jupyter notebook demo.ipynb

# C — run one fresh debate end-to-end against a live LLM
# (free-tier OpenRouter rate-limits; a paid key or LLM_MODEL=<paid-model>
# is the reliable path)
export OPENROUTER_API_KEY=...
python run_demo.py
```

What's in the repo: the four agent classes (`agents.py`), the orchestrator (`debate.py`), all the system prompts (`prompts/`), three real redacted production debates (`transcripts/`), the operational metrics behind the funnel (`operational_metrics.json`), and the walk-through notebook (`demo.ipynb`).

What's *not* in the repo: the downstream production pipeline, the backtest engine, and the actual factor formulas that the strategy trades. Those stay in the production codebase. The portfolio numbers above are evidence that the LLM debate system feeds something that works; they aren't reproducible from this scaffold alone.